

CPSC 304 - Milestone 4

Milestone #: ____4____

Date: ____April 1st, 2026____

Group Number: ____90____

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Kanaad Kalyan Bodkhe	57135345	b1x5s	bodkhekanaad@gmail.com
Saman Atiq	41874207	r0s7p	samanatiq004@gmail.com
Medhansh Choubey	50146505	d4e7x	medhanshchoubey619@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

1. Project Description

This project implements a desktop application for managing the daily operations of a law firm. The system is built using JavaFX for the graphical interface and Oracle SQL for persistent data storage, connected via JDBC over an SSH tunnel to UBC's department-provided Oracle servers.

The application allows users to manage court cases, lawyers (across three specializations: family, criminal, and corporate), clients, work logs, hearings, evidence, invoices, and courthouses. Specifically, the application supports:

- Inserting, updating, and deleting court case records with full error handling and success/failure feedback
- Searching for court cases using dynamic AND/OR filter conditions on any combination of attributes
- Viewing invoice records with user-selected and user-ordered columns
- Looking up cases by type and viewing the associated client and lawyer names via a three-way join
- Running six analytical queries over lawyer workload data: total hours grouped by lawyer, lawyers active on multiple cases, lawyers above average total hours, and lawyers who have logged work on every assigned case

All user inputs are handled via PreparedStatements to prevent SQL injection. The GUI is designed for non-technical users and contains no SQL syntax.

2. Database Schema

All constraints are listed exactly as implemented in setup.sql.

Client

Attribute	Type	Constraints
client_id	INTEGER	PRIMARY KEY
name	VARCHAR2(100)	NOT NULL
contact	VARCHAR2(100)	NOT NULL

amount_payable DECIMAL(10,2) —

Lawyer

Attribute	Type	Constraints
lawyer_id	INTEGER	PRIMARY KEY
bar_id	VARCHAR2(50)	UNIQUE, NOT NULL
name	VARCHAR2(100)	NOT NULL
start_date	DATE	—
new_cases	INTEGER	—
success_rate	DECIMAL(5,2)	—

CourtHouse

Attribute	Type	Constraints
courtid	INTEGER	PRIMARY KEY
judge_name	VARCHAR2(100)	—
city	VARCHAR2(50)	NOT NULL
province	VARCHAR2(50)	NOT NULL

Invoice_status

Attribute	Type	Constraints
status_code	NUMBER(1)	PRIMARY KEY
status_description	VARCHAR2(20)	—

Family

Attribute	Type	Constraints
lawyer_id	INTEGER	PRIMARY KEY, FK → Lawyer(lawyer_id) ON DELETE CASCADE
rate	DECIMAL(10,2)	—
mediation_certified	NUMBER(1)	—

Criminal

Attribute	Type	Constraints
lawyer_id	INTEGER	PRIMARY KEY, FK → Lawyer(lawyer_id) ON DELETE CASCADE
rate	DECIMAL(10,2)	—
security_clearance	VARCHAR2(50)	—

Corporate

Attribute	Type	Constraints
lawyer_id	INTEGER	PRIMARY KEY, FK → Lawyer(lawyer_id) ON DELETE CASCADE
rate	DECIMAL(10,2)	—
retainer_required	NUMBER(1)	—

Court_Case

Attribute	Type	Constraints
case_id	INTEGER	PRIMARY KEY
opening_date	DATE	NOT NULL
closing_date	DATE	—
case_type	VARCHAR2(50)	—
is_successful	NUMBER(1)	—
client_id	INTEGER	NOT NULL, FK → Client(client_id)
lawyer_id	INTEGER	NOT NULL, FK → Lawyer(lawyer_id)

WorkLog

Attribute	Type	Constraints
lawyer_id	INTEGER	NOT NULL, PK (part), FK → Lawyer(lawyer_id)
case_id	INTEGER	NOT NULL, PK (part), FK → Court_Case(case_id)
log_date	DATE	NOT NULL, PK (part)
number_of_hours	DECIMAL(5,2)	—

Primary Key: (lawyer_id, case_id, log_date)

Invoice

Attribute	Type	Constraints
invoice_id	INTEGER	PRIMARY KEY
amount	DECIMAL(10,2)	NOT NULL
case_id	INTEGER	UNIQUE, NOT NULL, FK → Court_Case(case_id)
status_code	NUMBER(1)	NOT NULL, FK → Invoice_status(status_code)

Hearing

Attribute	Type	Constraints
case_id	INTEGER	PK (part), FK → Court_Case(case_id) ON DELETE CASCADE
hearing_date	DATE	PK (part)
courtid	INTEGER	NOT NULL, FK → CourtHouse(courtid)

Primary Key: (case_id, hearing_date)

Evidence

Attribute	Type	Constraints
evidence_id	INTEGER	PRIMARY KEY
stored_in	VARCHAR2(50)	—
evidence_type	VARCHAR2(50)	—
case_id	INTEGER	NOT NULL, FK → Court_Case(case_id)

Presented

Attribute	Type	Constraints
evidence_id	INTEGER	PK (part), FK → Evidence(evidence_id)
case_id	INTEGER	PK (part)
hearing_date	DATE	PK (part)

Primary Key: (evidence_id, case_id, hearing_date) Foreign Key: (case_id, hearing_date) → Hearing(case_id, hearing_date)

3. Schema Changes from Milestone 2

The final schema is unchanged from the normalized schema submitted in Milestone 2. All 13 relations are implemented exactly as specified with no denormalization.

Note on Oracle FK limitation: Oracle does not support ON UPDATE CASCADE. Our implementation acknowledges that foreign key values referencing updated primary keys should be updated manually. No additional handling is implemented beyond this acknowledgment, as confirmed acceptable per the project spec.

4. Visual Representation of Database State

Client  				
#	client_id	name	contact	# amount_payable
	1	Alice Smith	alice@email.com	1500
	2	Bob Jones	bob@email.com	0
	3	Charlie Brown	charlie@email.com	450.5
	4	Diana Prince	diana@email.com	3200
	5	Evan Wright	evan@email.com	100

LawyerTable  						
	lawyer_id	bar_id	name	start_date	new_cases	success_rate
	101	BAR001	Harvey Specter	15-Jan-15	5	95.5
	102	BAR002	Saul Goodman	22-Mar-18	12	88
	103	BAR003	Kim Wexler	11-Jul-16	3	92
	104	BAR004	Matt Murdock	01-Sep-20	8	85.5
	105	BAR005	Alicia Florrick	05-Nov-12	2	98
	106	BAR006	James McGill	20-May-17	4	80
	107	BAR007	Elle Woods	15-Aug-19	6	91
	108	BAR008	Perry Mason	10-Mar-14	9	94
	109	BAR009	Jack McCoy	01-Jun-10	15	87.5
	110	BAR010	Olivia Benson	30-Sep-13	11	89
	111	BAR011	Mike Ross	12-Jan-21	2	76
	112	BAR012	Louis Litt	18-Apr-16	7	83
	113	BAR013	Jessica Pearson	22-Nov-08	20	97
	114	BAR014	Robert Zane	14-Jul-11	13	90.5
	115	BAR015	Rachel Zane	28-Feb-22	1	78

	A	B	C	D
	CourtHouseTable ▾ 			
	 courtid ▾	judge_name ▾	city ▾	province ▾
	1	Hon. Judy	Vancouver	BC
	2	Hon. Marshall	Burnaby	BC
	3	Hon. Thurgood	Richmond	BC
	4	Hon. Ginsburg	Surrey	BC
	5	Hon. Scalia	Victoria	BC

	InvoiceStatusTable ▾ 	
	status_code ▾	status_description ▾
	1	Pending
	2	Paid
	3	Overdue
	4	Cancelled
	5	Refunded

	A	B	C
	FamilyTable ▾		
	lawyer_id ▾	rate ▾	mediation_certified ▾
2	103	350	1
3	105	400	1
4	106	375	0
5	107	420	1
6	108	390	1

	A	B	C
	CriminalTable ▾		
	lawyer_id ▾	rate ▾	security_clearance ▾
	102	300	Level 2
	104	250	Level 1
	109	275	Level 3
	110	320	Level 2
	111	230	Level 1

CorporateTable		
lawyer_id	rate	retainer_required
101	800	1
112	750	0
113	950	1
114	880	1
115	700	0

CourtCaseTable						
case_id	opening_date	closing_date	case_type	is_successful	client_id	lawyer_id
1001	10-Jan-25		Family	0	1	103
1002	15-Feb-25	01-Mar-25	Criminal	1	2	102
1003	20-Nov-24		Corporate	0	4	101
1004	01-Mar-25		Criminal	0	3	104
1005	15-Jun-24	10-Jan-25	Family	1	5	105
1006	01-Apr-25		Corporate	0	1	101
1007	10-May-25		Criminal	0	2	102
1008	01-Jun-25		Family	0	3	103
1009	15-Jul-25	01-Sep-25	Criminal	1	4	104
1010	20-Aug-25		Family	0	5	105
1011	01-Mar-25	15-Jun-25	Corporate	1	2	112
1012	10-Apr-25	30-Aug-25	Family	1	3	106

	A	B	C	D
	WorkLogTable 			
1	 lawyer_id 	case_id 	log_date 	number_of_hours 
2	103	1001	11-Jan-25	4.5
3	102	1002	16-Feb-25	2
4	101	1003	21-Nov-24	10
5	104	1004	02-Mar-25	6.5
6	105	1005	16-Jun-24	3
7	101	1006	02-Apr-25	8
8	101	1003	10-Apr-25	5
9	102	1002	05-Mar-25	3
10	103	1008	02-Jun-25	3.5
11	103	1001	15-Feb-25	4
12	104	1009	16-Jul-25	5
13	105	1010	21-Aug-25	4
14				

A	B	C	D
InvoiceTable 			
			
invoice_id 	amount 	case_id 	status_code 
5002	600	1002	2
5005	1350	1005	2
5009	875	1009	2
5011	2200	1011	2
5012	1100	1012	1

HearingTable  		
case_id 	hearing_date 	courtid 
1001	01-Feb-25	1
1002	20-Feb-25	2
1003	15-Jan-25	1
1004	15-Mar-25	3
1005	10-Sep-24	4
1006	01-May-25	2
1007	15-Jun-25	3
1008	10-Jul-25	4
1009	05-Aug-25	5
1010	20-Sep-25	1

EvidenceTable  				
 evidence_id 	stored_in 	evidence_type 	case_id 	
901	Locker A	Custody Agreement	1001	
902	Locker B	Security Footage	1002	
903	Server	Email Thread	1003	
904	Locker C	Weapon	1004	
905	Digital	Text Messages	1005	
906	Locker D	Contract	1006	
907	Digital	CCTV Footage	1007	
908	Locker E	Birth Certificate	1008	
909	Server	Financial Report	1009	
910	Digital	Voice Recording	1010	

PresentedTable		
evidence_id	case_id	hearing_date
901	1001	01-Feb-25
902	1002	20-Feb-25
903	1003	15-Jan-25
904	1004	15-Mar-25
905	1005	10-Sep-24
906	1006	01-May-25
907	1007	15-Jun-25
908	1008	10-Jul-25
909	1009	05-Aug-25
910	1010	20-Sep-25

After running setup.sql the database contains:

- Client: 5 rows (client_ids 1–5)
- Lawyer: 15 rows (lawyer_ids 101–115)
- CourtHouse: 5 rows (courtids 1–5)
- Invoice_status: 5 rows (status_codes 1–5)
- Family: 5 rows (lawyer_ids 103, 105, 106, 107, 108)
- Criminal: 5 rows (lawyer_ids 102, 104, 109, 110, 111)
- Corporate: 5 rows (lawyer_ids 101, 112, 113, 114, 115)
- Court_Case: 12 rows (case_ids 1001–1012)
- WorkLog: 12 rows
- Invoice: 5 rows (invoice_ids 5002, 5005, 5009, 5011, 5012)
- Hearing: 10 rows (one per case, cases 1001–1010)
- Evidence: 10 rows (evidence_ids 901–910)
- Presented: 10 rows

5. SQL Query List File Names and Line Numbers

Query 1 — INSERT

Relation: Court_Case (references Client via client_id FK and Lawyer via lawyer_id FK) **File:** `src/main/java/backend/controller/MainController.java` **Lines:** 131–134

SQL:

```
INSERT INTO Court_Case (case_id, case_type, client_id, lawyer_id, opening_date)
VALUES (?, ?, ?, ?, SYSDATE)
```

The user enters Case ID, Case Type, Client ID, and Lawyer ID via the Court Case Management screen. All values are bound via PreparedStatement. If the Client ID or Lawyer ID does not exist in the referenced table, Oracle raises ORA-02291 and the application displays: "Client ID or Lawyer ID does not exist in the system." Duplicate Case IDs trigger ORA-00001 with the message: "A case with that ID already exists."

Query 2 — UPDATE

Relation: Court_Case **Non-PK attributes updatable:** case_type, client_id (FK → Client), lawyer_id (FK → Lawyer) **Satisfies:** 2+ non-PK attributes; at least one (client_id, lawyer_id) is a FK **File:** `src/main/java/backend/controller/MainController.java` **Lines:** 183–196

SQL (dynamically constructed at runtime):

```
UPDATE Court_Case SET <user-selected fields> WHERE case_id = ?
```

The SET clause is built at runtime based on which fields the user fills in (case_type, client_id, lawyer_id). Any combination of one or more fields is valid. The current table is displayed above the form and clicking a row pre-fills the Case ID field. All values bound via PreparedStatement.

Query 3 — DELETE

Relation: Court_Case **Cascade:** Deleting a Court_Case row automatically deletes all Hearing rows referencing it via ON DELETE CASCADE defined on Hearing.case_id in setup.sql **File:** `src/main/java/backend/controller/MainController.java` **Lines:** 233–234

SQL:

```
DELETE FROM Court_Case WHERE case_id = ?
```

The user enters a Case ID. If no matching row exists the application shows: "No case found with ID X." If the case has dependent records that are not cascade-deleted (e.g. Invoice), Oracle raises ORA-02292 and the application shows: "Cannot delete — this case is referenced by other records."

Query 4 — Selection

Relation: Court_Case **File:** `src/main/java/backend/repository/CaseRepo.java` **Lines:** 35–36 (base query string), 21–97 (full method)

SQL (dynamically constructed at runtime):

```
SELECT case_id, case_type, client_id, lawyer_id
FROM Court_Case
WHERE <user-defined conditions joined by AND / OR>
```

The user adds any number of conditions via the Case Search screen. Each condition specifies an attribute (chosen from a dropdown: case_id, case_type, client_id, lawyer_id, is_successful, closing_date), an operator (chosen from a dropdown: =, <, >, <=, >=, <>, LIKE, IS NULL, IS NOT NULL), and a value. Conditions are joined with AND or OR as chosen per condition by the user. All values are bound via PreparedStatement parameters; attributes and operators come from controlled dropdown inputs, not free text, preventing SQL injection.

Query 5 — Projection

Relation: Invoice **File:** `src/main/java/backend/repository/InvoiceRepo.java` **Line:** 28

SQL (dynamically constructed at runtime):

```
SELECT <user-selected columns in user-specified order> FROM Invoice
```

The user selects any subset of columns (invoice_id, amount, case_id, status_code) via checkboxes on the Invoice View screen and reorders them using Up/Down buttons. Column names are drawn exclusively from a fixed allowed list — no user-typed string is interpolated into the query. The order of columns in the SELECT clause matches the user-specified order exactly; no client-side reordering is performed.

Query 6 — Join

Relations joined: Court_Case, Client, Lawyer (3 relations) **File:** `src/main/java/backend/repository/LawyerRepo.java` **Lines:** 22–30

SQL:

```
SELECT cc.case_id, cc.case_type,
       cc.opening_date, cc.closing_date,
       c.name AS client_name,
       l.name AS lawyer_name
FROM Court_Case cc
JOIN Client c ON cc.client_id = c.client_id
JOIN Lawyer l ON cc.lawyer_id = l.lawyer_id
WHERE cc.case_type = ?
```

The user selects a case type from a dropdown (Family, Criminal, or Corporate) via the Case-Lawyer Lookup screen. The value is bound via PreparedStatement. Results show each matching case with the full names of the assigned client and lawyer.

Query 7 — Aggregation with GROUP BY

Relations: Lawyer, WorkLog **File:**

`src/main/java/backend/repository/WorkLogRepo.java` **Lines:** 22–29

SQL:

```
SELECT l.lawyer_id, l.name,
       COUNT(wl.case_id) AS log_entries,
       SUM(wl.number_of_hours) AS total_hours
FROM Lawyer l
JOIN WorkLog wl ON l.lawyer_id = wl.lawyer_id
GROUP BY l.lawyer_id, l.name
ORDER BY total_hours DESC
```

Description: For each lawyer, this query counts the total number of work log entries and sums the total hours logged across all their cases, grouping results by lawyer and ordering from highest to lowest total hours.

Query 8 — Aggregation with HAVING

Relations: Lawyer, WorkLog **File:**

`src/main/java/backend/repository/WorkLogRepo.java` **Lines:** 57–64

SQL:

```
SELECT l.lawyer_id, l.name,
       COUNT(DISTINCT wl.case_id) AS cases_worked
FROM Lawyer l
JOIN WorkLog wl ON l.lawyer_id = wl.lawyer_id
GROUP BY l.lawyer_id, l.name
HAVING COUNT(DISTINCT wl.case_id) > 1
ORDER BY cases_worked DESC
```

Description: Groups lawyers by their logged work and uses a HAVING clause to return only those who have logged work on more than one distinct case, along with the count of distinct cases each qualifying lawyer has worked on.

Query 9 — Nested Aggregation with GROUP BY

Relations: Lawyer, WorkLog **File:**

`src/main/java/backend/repository/WorkLogRepo.java` **Lines:** 92–106

SQL:

```
SELECT l.lawyer_id, l.name,
       SUM(wl.number_of_hours) AS total_hours
FROM Lawyer l
JOIN WorkLog wl ON l.lawyer_id = wl.lawyer_id
GROUP BY l.lawyer_id, l.name
HAVING SUM(wl.number_of_hours) >= (
    SELECT AVG(lawyer_total)
    FROM (
        SELECT SUM(number_of_hours) AS lawyer_total
        FROM WorkLog
        GROUP BY lawyer_id
    )
)
ORDER BY total_hours DESC
```

Description: Returns each lawyer whose total logged hours meets or exceeds the average total hours across all lawyers; the inner subquery computes each lawyer's total hours, the outer subquery averages those per-lawyer totals to produce the threshold, and the outer query uses GROUP BY and HAVING to compare each lawyer's sum against that threshold.

Query 10 — Division

Relations: Lawyer, Court_Case, WorkLog **File:**

`src/main/java/backend/repository/LawyerRepo.java` **Lines:** 64–76

SQL:

```
SELECT l.lawyer_id, l.name
FROM Lawyer l
```

```
WHERE NOT EXISTS (  
    SELECT cc.case_id  
    FROM Court_Case cc  
    WHERE cc.lawyer_id = l.lawyer_id  
    MINUS  
    SELECT wl.case_id  
    FROM WorkLog wl  
    WHERE wl.lawyer_id = l.lawyer_id  
)  
ORDER BY l.lawyer_id
```

Description: Returns lawyers who have logged work on every case assigned to them; relational division is implemented using NOT EXISTS with MINUS (Oracle's equivalent of EXCEPT), where a lawyer is excluded if the set of their assigned cases minus the set of cases they have logged work on is non-empty.

- Note : We used grammarly for grammar help.